```
$ minimalterminal slides.html
```

How MinimalTerminal works

Five steps from a hand-written slide file to a slide on the terminal screen.

PARSER → STYLER → LAYOUT → RENDERER → PRESENTER

THE PIPELINE

Five steps, each feeding the next

1

Parser

Reads the slide file and builds a tree of nodes.

2

Styler

Decides how each node looks, using the theme.

3

Layout

Decides where each node goes on the screen.

4

Renderer

Draws the result in the terminal.

5

Presenter

Waits for keys and moves between slides.

Each step takes the previous step's output, so the parser's tree is what everything else is built on.

THE INPUT

One slide to trace through every step

We write slides by hand in a small set of HTML tags. This file is the example for the rest of the deck.

slides.html

```
<deck>
  <slide>
    <h1>Hello</h1>
    <p>This is <b>bold</b> text</p>
  </slide>
</deck>
```

What a terminal gives us

- A grid of character cells, for example 80 columns by 24 rows
- One character per cell, with a text color and a background color
- Bold, italic and underline
- No font or font size changes

STEP 1 OF 5

Parser

Turns the text of the file into a tree that says what is inside what.

What it does

- Reads the slide file as plain text
- Picks out the supported tags and the text between them
- Makes each one a node and nests it under its parent
- Knows nothing about colors or screen positions

```
Output: a tree of nodes

Deck
└─ Slide
   ├─ H1
   │  └─ Text "Hello"
   └─ P
      ├─ Text "This is "
      ├─ B
      │  └─ Text "bold"
      └─ Text " text"
```

STEP 2 OF 5

Styler

Works out how every node should look, using the theme file.

What it does

- Reads the rules in theme.css
- Attaches a look to each node in the tree
- Lets children inherit from their parents
- The B node has no color of its own, so it takes white from P and adds bold

```
theme.css (example rules)

h1 { color: cyan; font-weight: bold }
p  { color: white }

Output: the same tree, with a look on each node

H1 → cyan, bold
P  → white
B  → white (from P), bold
```

STEP 3 OF 5

Layout

Decides where each node goes on the grid of character cells.

What it does

- Takes the styled tree and the terminal's current size
- Gives each node a row, a column and a width
- Handles word wrapping, alignment, padding and margins
- Runs again when the window is resized

```
Output: a position for each node (80 × 24 terminal)

H1 "Hello"
  row 2, centered → column 38

P "This is bold text"
  row 5, column 4
  wraps if longer than the width
```

STEP 4 OF 5

Renderer

Draws the positioned nodes in the terminal.

What it does

- Fills a grid of cells from the layout's positions
- Writes the grid to the terminal as ANSI escape codes
- Escape codes set colors and bold, and move the cursor
- Switches the terminal to raw mode so keys can be read one at a time

```
Output: characters and escape codes

\x1b[2;38H
  move the cursor to row 2, column 38

\x1b[1;36mHello\x1b[0m
  bold cyan on, print Hello, reset
```

STEP 5 OF 5

Presenter

The loop that keeps the program running and moves through the deck.

What it does

- Waits for a key press
- Updates the current slide: next, back or quit
- Asks Layout and Renderer to redraw
- If bullets appear one at a time, tracks the slide and the step together

The input loop (example keys)

```
repeat:
  wait for a key
  right arrow: next slide
  left arrow:  previous slide
  q:           quit
  redraw the current slide
```

PUTTING IT TOGETHER

When each step runs

ONCE, AT LOAD

Parser and Styler

The file is read and styled one time when the program starts.

ON EVERY CHANGE

Layout and Renderer

They run again when the slide changes or the window is resized.

THE WHOLE TIME

Presenter

It waits for keys and triggers each redraw until you quit.

Every later step reads the parser's tree, so we agree on the node format before writing code.